

CLEAN ARCHITECTURE · BLOQUE 3

Solución guiada del reto

Sistema de aprobación de gastos

TypeScript · Bun

BIG school

No comparamos árboles de carpetas

Comparamos quién toma cada decisión y hacia dónde apuntan las dependencias.

La estructura ayuda a leer; el comportamiento y los imports cuentan la verdad.

Cuatro clases de decisión

CREAR

Estado inicial

¿Queda pendiente o nace resuelto?

TRANSICIÓN

Cambiar estado

¿Quién puede aprobar o rechazar?

COORDINAR

Orquestar

¿Qué se carga, guarda y notifica?

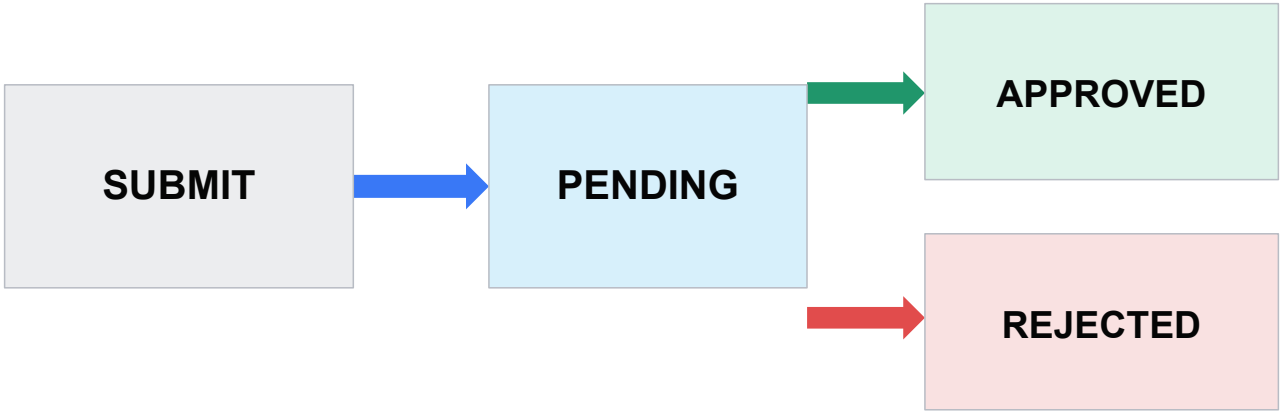
TRADUCIR

Cruzar el borde

¿Cómo llega HTTP y qué devuelve?

BIG school

El ciclo de vida es explícito



Las decisiones automáticas pueden nacer ya resueltas.

BIG school

Una responsabilidad por frontera

DOMAIN

Reglas

Estados, política e invariantes.

APPLICATION

Secuencia

Ports, casos de uso y efectos.

ADAPTERS

Detalles

Memoria, HTTP, reloj e IDs.

COMPOSITION

Elección

Implementaciones conectadas.

BIG school

La solución avanza en ocho etapas

01

Reglas

Ejemplos

02

Dominio

Expense

03

Submit

Ports

04

Decidir

Manual

05

Adapters

Efectos

06

HTTP

Borde

07

Montar

Demo

08

Verificar

Producción

BIG school

El estado es una unión discriminada

```
type ExpenseState =
  | { type: "pending"; requiredRole: ApprovalRole }
  | { type: "approved"; decision: Decision }
  | { type: "rejected"; decision: Decision };
```

BIG school

La política respeta precedencia

1

Personal

Siempre rejected por system.

2

≤ 100 €

Approved automáticamente.

3

≤ 1.000 €

Pending para manager.

4

> 1.000 €

Pending para finance.

BIG school

Una transición devuelve otra entidad

```
approve(actor): Expense {
  this.assertCanResolve(actor);

  return new Expense({
    ...this.props,
    state: { type: "approved", decision:
approvedBy(actor) },
  });
}
```

BIG school

Validar en un orden estable

01

Resuelto

No repetir transición.

02

Propietario

No decidir lo propio.

03

Rol

Debe coincidir.

04

Motivo

Obligatorio al rechazar.

BIG school

Los tests fijan la política

Creación

Personal → rejected
≤ 100 € → approved
600 € → pending manager
1.500 € → pending finance

Transición

Rol correcto
Sin autoaprobación
No resolver dos veces
Motivo de rechazo

BIG school

SubmitExpense define una frontera

Entrada

employeeId
amountInCents
category
submittedOn

Salida

ExpenseView

Estado explícito
y estable

BIG school

Cuatro ports, cuatro necesidades

STATE

Repository

Guardar y recuperar Expense.

TIME

Clock

Fecha de la decisión.

IDENTITY

IdGenerator

Nuevo ExpenseId.

EFFECT

Notification

Comunicar resolución.

BIG school

SubmitExpense coordina seis movimientos

01	02	03	04	05	06
Validar	**Identificar**	**Crear**	**Guardar**	**Notificar**	**Mapear**
Input	ID	Expense	Estado	Si resuelto	View

BIG school

Notificamos solo al resolver

```
await repository.save(expense);

if (expense.isResolved()) {
  await notifyResolvedExpense(expense, notifications);
}

return toExpenseView(expense);
```

BIG school

Los fakes controlan la aplicación

Determinismo

FixedClock
FixedIdGenerator
InMemoryExpenseRepository

Observabilidad

RecordingNotificationGateway

Datos, llamadas y orden visibles

BIG school

Tres casos de uso, un modelo

SubmitExpense · ApproveExpense · RejectExpense

Cada intención tiene un contrato pequeño; las reglas convergen en Expense.

La decisión manual tiene cinco pasos

01	02	03	04	05
Cargar	**Resolver**	**Decidir**	**Guardar**	**Notificar**
Expense	Not found	Dominio	Nuevo estado	Resultado

BIG school

Pequeña duplicación, intención clara

ApproveExpense

Carga
aprueba
guarda
notifica

RejectExpense

Carga
rechaza con motivo
guarda
notifica

BIG school

El motivo pertenece al dominio

Una Expense rechazada sin reason es un estado inválido.

Validarlo en el controller permitiría otros caminos inconsistentes.

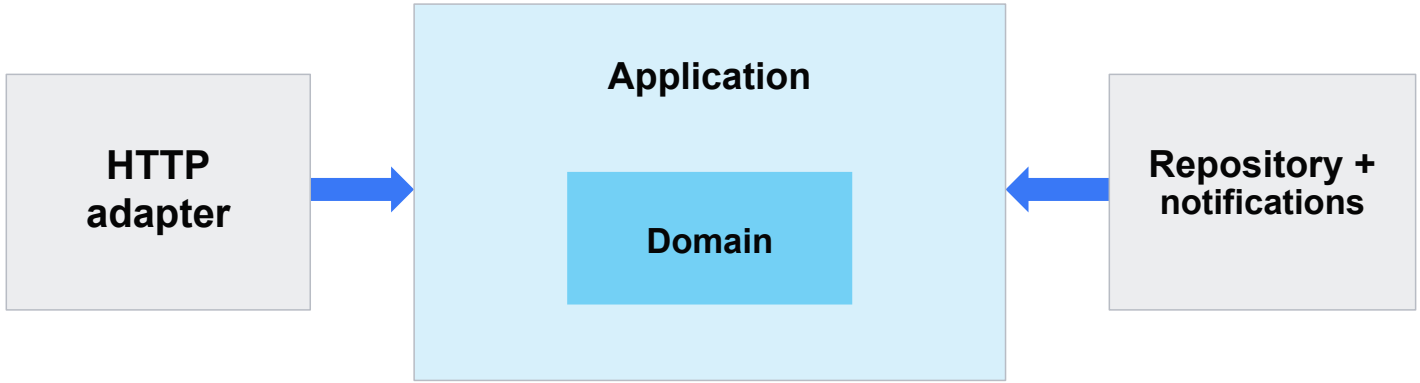
Un rechazo no produce efectos

```
const result = await useCase.execute(input);

expect(result).toEqual({ ok: false, reason: "forbidden" });
expect(repository.saved).toHaveLength(0);
expect(notifications.sent).toHaveLength(0);
```

BIG school

Los adapters implementan ports



Las flechas de dependencia apuntan hacia las reglas y los contratos.

BIG school

Memoria conserva entidades inmutables

Al guardar

Map<ExpenseId, Expense>

Se reemplaza la referencia por la nueva entidad.

Al recuperar

Se devuelve Expense, no una fila ni un DTO de infraestructura.

BIG school

La notificación es observable

Port

notifyApproved(view)
notifyRejected(view)

Recording adapter

sent: lista de Notification

Permite verificar contenido y orden.

BIG school

Guardar y notificar no es atómico

save() ✓ notify() × → estado persistido sin mensaje

En producción: transaction + outbox + worker idempotente.

HTTP termina en el borde

Validar forma. Invocar intención. Traducir resultado.

El controller no vuelve a decidir la política de gastos.

BIG school

Unknown obliga a validar

```
function parseBody(body: unknown): SubmitExpenseInput |
null {
  if (!isRecord(body) || typeof body.employeeId !==
"string") {
    return null;
  }

  return { employeeId: body.employeeId, ... };
}
```

BIG school

Cada resultado tiene semántica HTTP

400	403	404	409	200/201
Invalid	**Forbidden**	**Not found**	**Conflict**	**Success**
Forma o datos.	Actor o rol.	Expense.	Ya resuelto.	Intención aplicada.

BIG school

Traducción común, intenciones separadas

Reutilizable

toErrorResponse(result)
parseCommonFields(body)

Separado

SubmitController
ApproveController
RejectController

BIG school

El controller se prueba como traductor

```
const response = await controller.handle({ body });

expect(useCase.received).toEqual(expectedInput);
expect(response).toEqual({
  status: 403,
  body: { error: "forbidden" },
});
```

BIG school

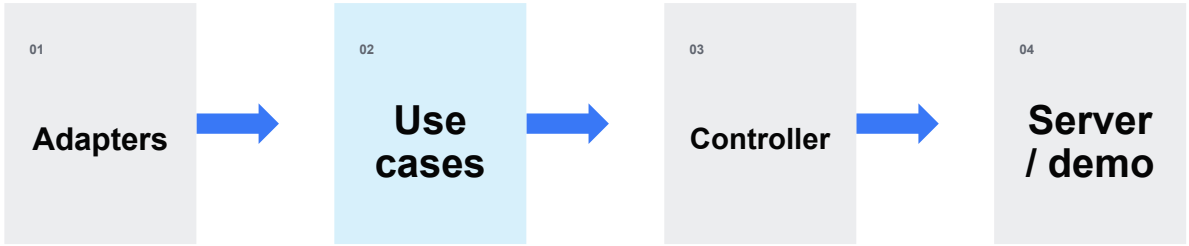
Componer también es diseñar

Elegimos detalles concretos en un único borde de la aplicación.

Ningún caso de uso construye sus propias dependencias.

BIG school

El object graph se lee de izquierda a derecha



Construir de dentro hacia fuera mantiene visibles todas las elecciones.

La demo hace visible el recorrido

```
$ bun run start

Submitted: expense-1 · pending · manager
Approved:  expense-1 · approved · manager-7
Notifications: 1
```

BIG school

Cambiar el framework no cambia el caso

Hoy

Minimal HTTP types
In-memory repository
Recording notifications

Mañana

Hono / Fastify
PostgreSQL
Email / queue

Tests por nivel

DOMAIN

¿Decide bien?

Política e invariantes.

APPLICATION

¿Coordina bien?

Secuencia y efectos.

ADAPTER

¿Traduce bien?

Ports y tecnología.

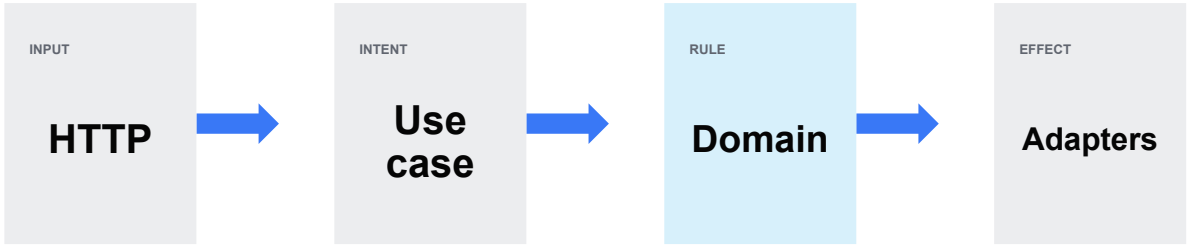
ACCEPTANCE

¿Encaja todo?

Corte vertical completo.

BIG school

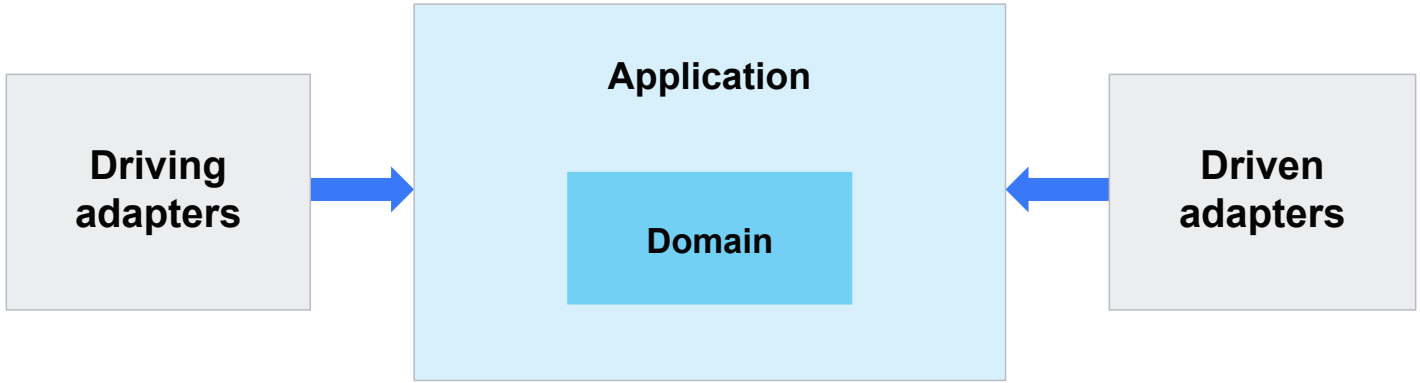
La aceptación recorre un corte vertical



Un escenario real verifica contrato, decisión y efecto observable.

BIG school

Dependencias hacia dentro



Carpetas orientan; imports, constructores y tests demuestran la arquitectura.

BIG school

Qué falta para producción

DATA

PostgreSQL

Transacciones y concurrencia.

MESSAGING

Outbox

Entrega fiable e idempotencia.

SECURITY

AuthN / AuthZ

Identidad real y permisos.

OPS

Observabilidad

Logs, métricas y trazas.

BIG school

La IA propone; el equipo decide

Útil para

Explorar contratos
Generar casos
Detectar duplicación
Proponer tests

Requiere juicio

Límites
Precedencia
Consistencia
Trade-offs

BIG school

La arquitectura es una cadena de decisiones

Regla → modelo → port → adapter → composición → prueba

Si cada flecha puede explicarse, la solución es defendible y mantenible.

BIG school